05. LINUX

Linux Command

핵심 명령어로 리눅스 시스템을 자유자재로 다루기

1

명령어 기본 구조

명령어를 실행하려면, 명령어 이름과 인자를 커맨드라인에 입력합니다.

```
$ ls  -l  /etc
```

1

명령어 이름 (command)

실행할 프로그램의 이름

2

옵션 (options / flags)

동작 방식을 바꾸는 -로 시작하는 값

3

인자 (arguments)

명령어가 작업할 대상(파일·경로 등)

도움말 보기: man

리눅스의 HELP에 해당하는 것이 man (manual)입니다.

- man <명령어> 로 해당 명령어의 도움말을 확인합니다.
- 출력은 한 페이지씩 보여집니다

예시

```
$ man ls
```

TIP man 페이지가 없는 명령어는 --help 옵션으로 간단한 사용법을 확인할 수 있습니다 (예: ls --help)

man 페이지 탐색 키

`b`	이전 페이지로 이동
`f` 혹은 `space`	다음 페이지로 이동
`q`	man 종료
`/검색어`	본문 내 검색

2

권한 상승: sudo

관리자 권한이 필요한 작업은 sudo를 앞에 붙여 실행합니다.

> **안내** AWS EC2의 기본 사용자(ec2-user 등)는 root 비밀번호가 없고 sudo 그룹에 속해 있어, sudo로만 관리자 권한을 사용할 수 있습니다.

sudo 사용 예시

```
$ sudo dnf install httpd
# 관리자 권한으로 httpd 설치

$ sudo -i
# root 사용자로 전환 (비밀번호 없이 가능)
```

3

표준 입출력 스트림

프로세스는 세 가지 표준 스트림을 통해 터미널과 데이터를 주고받습니다.

#0 **`stdin`**

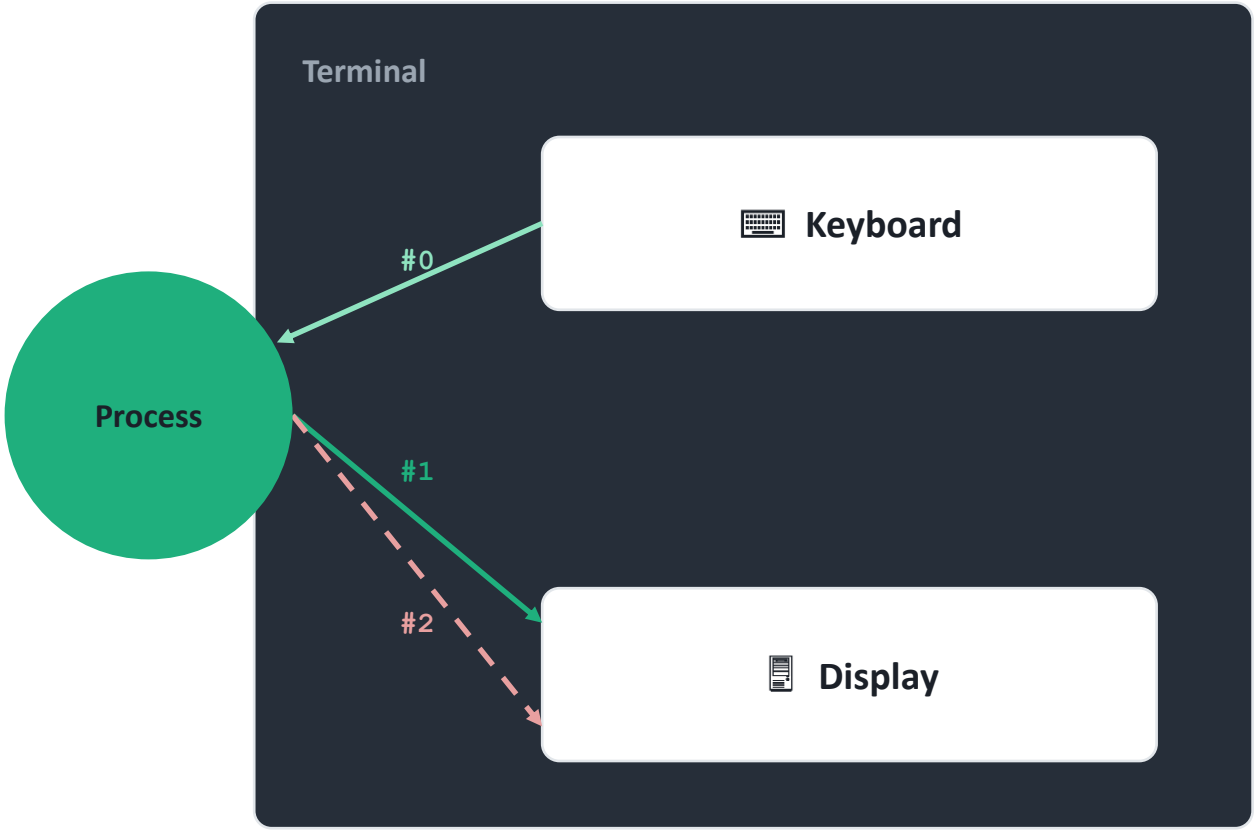
명령어가 입력을 받는 곳 — 기본값은 터미널(키보드)

#1 **`stdout`**

명령어가 결과를 출력하는 곳 — 기본값은 터미널(화면)

#2 **`stderr`**

명령어가 오류 메시지를 출력하는 곳 — 기본값은 터미널(화면)



출력 리다이렉션 (>, 2>)

명령어의 출력을 파일로 보낼 수 있습니다 — ">"로 출력 파일을 지정 (1>과 동일).

리다이렉션 예제

```
$ ls -l > output.txt
# 표준출력을 output.txt로 저장

$ javac HelloWorld.java 2> error.txt
# HelloWorld.java를 컴파일하고, 에러 메시지만 error.txt에 저장

$ javac HelloWorld.java > output.txt 2> error.txt
# 표준출력은 output.txt, 표준에러는 error.txt로 각각 분리 저장
```

입력 리다이렉션과 파이프

입력 리다이렉션 (<)

```
$ wc -l < input.txt
# input.txt를 입력으로 받아 줄 수를 출력
```

파이프 (|) — 한 명령의 출력을 다른 명령의 입력으로

```
$ ps -ef | grep httpd | wc -l
# ps 결과 → grep으로 "httpd" 포함 줄만 추림 → wc로 그 줄 수를 셈
# 결과: httpd 관련 프로세스 개수 출력
```

4

기본 명령어: pwd · who · whoami · which · su

기본 명령어

```
$ pwd
/home/ec2-user

$ whoami
ec2-user

$ which whoami
/usr/bin/whoami
```

`pwd`	현재 작업 디렉터리(print working directory) 출력
`whoami`	현재 로그인한 사용자 이름
`which`	명령어의 전체 경로 표시
`who`	현재 시스템에 로그인한 모든 사용자 목록 표시
`su - <계정명>`	계정 전환. exit로 이전 계정 복귀 (예: su - root)

디렉터리 구조 기호와 이동 (cd)

기호	의미	예시
/	루트 디렉터리	최상위 디렉터리
.	현재 디렉터리	./script.sh
..	현재 위치 기준 상위(부모) 디렉터리	cd ..
~	홈 디렉터리	cd ~
-	바로 이전 디렉터리 (cd 전용)	cd -

cd 예시

```
$ cd ~ && pwd
/home/ec2-user
```

5

파일 명령어: cp · mv · touch · rm · mkdir

파일·디렉터리 다루기

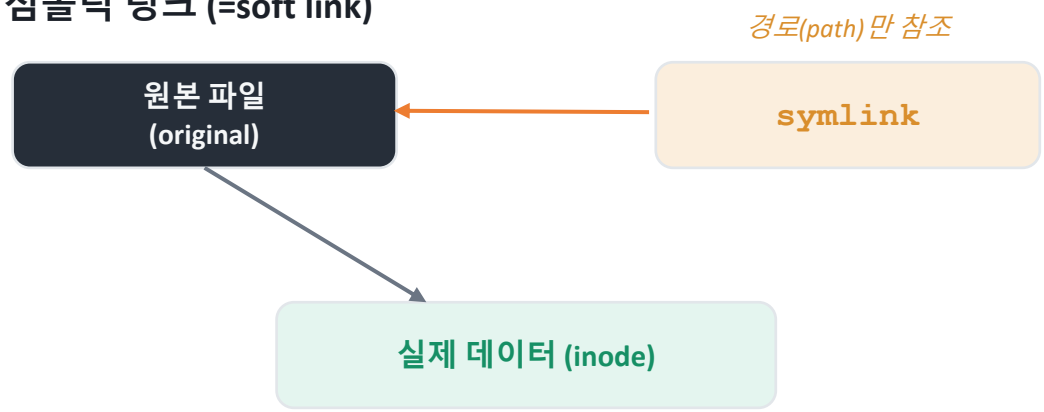
```
$ cp fromfile tofile
# fromfile을 tofile로 복사
$ mv fromfile tofile
# fromfile을 tofile로 이동/이름변경
$ touch sample.txt
# 빈 파일 생성
$ rm sample.txt
# 파일 삭제
$ rm -r somedir
# somedir 안의 모든 파일·디렉터리까지 재귀적으로 삭제
$ mkdir newdir
# 새 디렉터리 생성
$ rmdir olddir
# 비어 있는 디렉터리만 삭제
```

링크: 심볼릭 링크 vs 하드 링크

심볼릭 링크 생성 (symlink가 이미 있으면 실패)

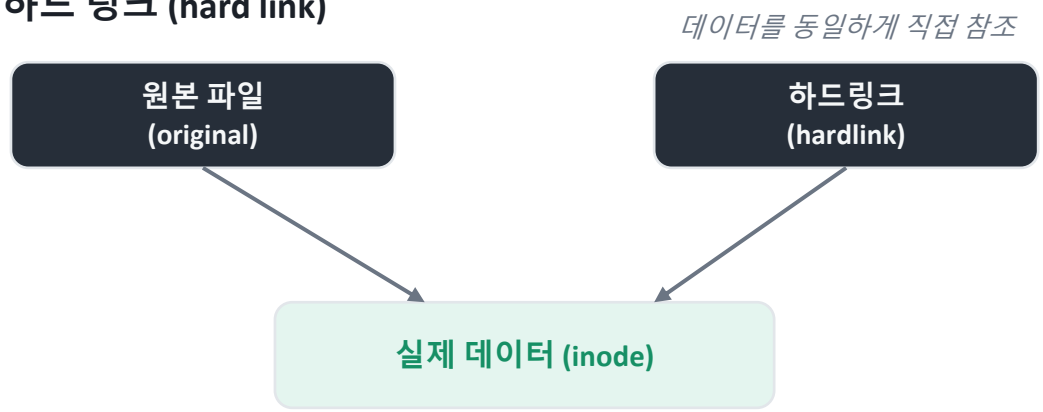
```
$ ln -s /path/file /path/symlink
```

심볼릭 링크 (=soft link)



- 원본 파일의 "경로"를 가리키는 바로가기
- 원본을 삭제하면 경로가 끊어져 symlink가 깨짐 (broken link)
- 디렉터리에도 만들 수 있고, 다른 파일시스템의 파일도 가리킬 수 있음

하드 링크 (hard link)



- 원본과 하드링크 모두 같은 데이터(inode)를 직접 가리킴
- 원본을 삭제해도 하드링크 이름이 남아있으면 데이터는 유지됨
- 디렉터리에는 만들 수 없고, 같은 파일시스템 안에서만 가능

파일 내용 보기: cat · more · less · head · tail

파일 보기 명령어

```
$ cat sample.txt
# 파일 내용을 표준출력으로 그대로 출력
$ more sample.txt
# 한 페이지씩 보기, 스페이스바로 다음 페이지
$ less sample.txt
# more과 유사하지만 위아래 모두 이동 가능 (f/b, j/k, /검색, q종료)
$ head sample.txt
# 파일의 처음 10줄 표시 (head -n N: N줄)
$ tail -f /var/log/messages
# 파일 끝부분을 실시간으로 계속 보여줌 (로그 모니터링에 유용)
```

6

ls와 파일 권한 개요

ls -al

```
$ ls -al
# -a: 숨김 파일 포함, -l: 상세 정보(long format)
drwxr-xr-x  2 ec2-user ec2-user  4096  Sep 10 10:00 .
-rw-r--r--  1 ec2-user ec2-user   220  Sep 10 10:00 sample.txt
```

핵심 맨 앞 10글자가 파일 종류(d=디렉터리, -=파일)와 권한(rwx)을 나타냅니다 — 다음 슬라이드에서 자세히 설명합니다.

권한 상세: User · Group · Other

모든 파일·디렉터리는 소유자(User) · 그룹(Group) · 그 외(Other) 각각에 대해 rwx 권한을 가집니다.

User			Group			Other		
R	W	X	R	W	X	R	W	X
4	2	1	4	2	1	4	2	1

파일 권한

read (r) 파일 내용을 화면에서 확인 (cat, less 등)
write (w) 파일 내용을 수정 · 삭제
execute (x) 파일을 프로그램 · 스크립트로 실행

디렉터리 권한

read (r) ls로 디렉터리 안의 파일 목록을 확인
write (w) 디렉터리 안에 파일 · 폴더를 생성 · 삭제
execute (x) cd 뿐만 아니라, 그 디렉터리를 통과할 수 있는 권한

TIP 디렉터리는 read(목록 보기)와 execute(통과 · 접근)가 서로 다른 권한입니다 — x 없이 r만 있으면 목록은 보여도 안에 들어가거나 파일을 열 수 없습니다.

chmod — 권한 변경

파일·디렉터리의 접근 권한을 변경합니다.

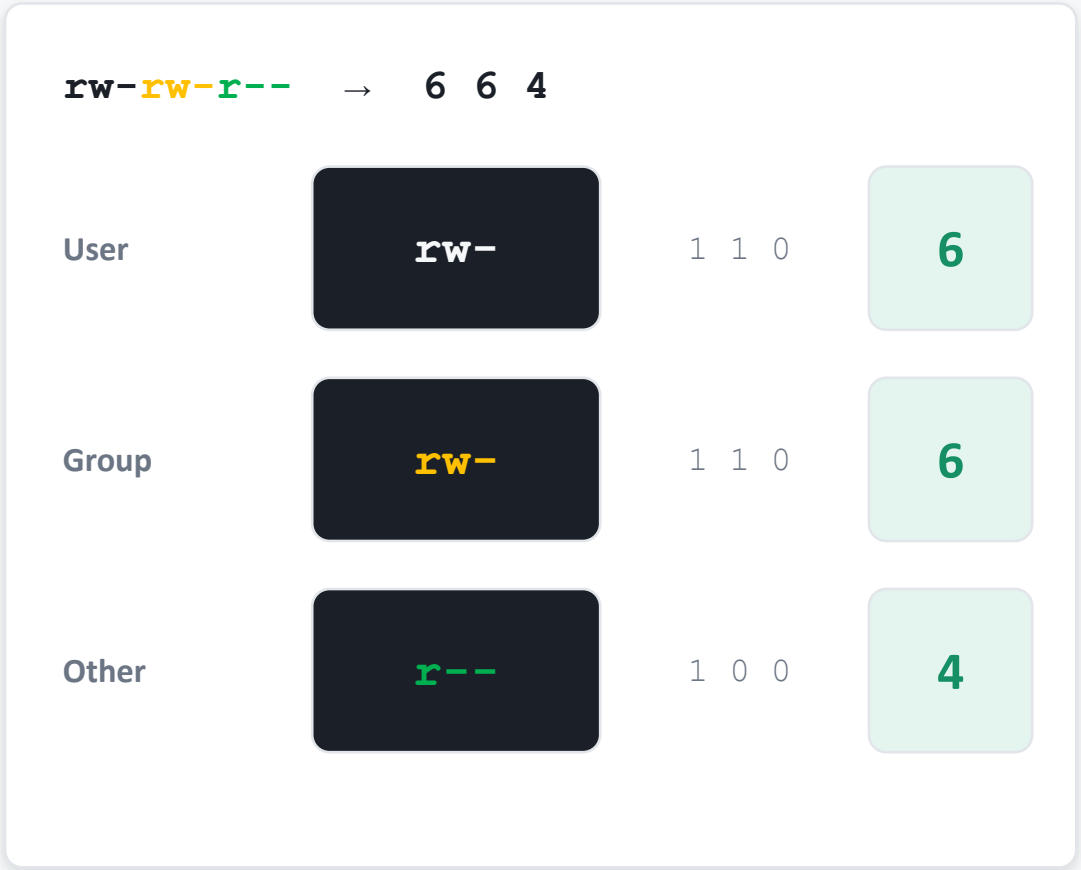
chmod 예시

```
$ chmod a+x filename
# 소유자·그룹·기타 모두에게 실행 권한 부여 (a=all)

$ chmod u-rx filename
# 소유자(u)의 읽기·실행 권한 제거
```

숫자 모드

```
$ chmod 664 filename
# 숫자 모드: 소유자·그룹은 읽기+쓰기(6), 기타는 읽기만(4)
```



chown · chgrp — 소유자 변경

chmod가 "권한"을 바꾼다면, chown/chgrp는 파일의 "소유자·그룹" 자체를 바꿉니다.

chown · chgrp

```
$ sudo chown ec2-user sample.txt
# 소유자를 ec2-user로 변경

$ sudo chown ec2-user:webteam sample.txt
# 소유자와 그룹을 동시에 변경 (사용자:그룹)

$ sudo chgrp webteam sample.txt
# 그룹만 webteam으로 변경
```

TIP 소유자·그룹 변경은 관리자 권한(sudo)이 필요한 경우가 대부분입니다.

7

find — 파일 검색

디렉터리 안에서 조건에 맞는 파일을 검색합니다.

find 예시

```
$ find . -name "*.java"
# 현재 디렉터리 이하에서 이름이 .java로 끝나는 파일 검색

$ find /var/log -type f -mtime -1
# /var/log에서 최근 1일 이내 수정된 일반 파일 검색

$ find . -size +100M
# 현재 위치 이하에서 100MB보다 큰 파일 검색
```

-mtime -1 (0~24시간)	-mtime 1 (24~48시간)	-mtime +1 (48시간~)

grep — 패턴 검색 기본

`grep [옵션] 패턴 [파일...]` — 파일에서 특정 패턴이 포함된 줄을 찾아 출력

access.log에서 IP 검색

```
$ grep "192.168.1.1" access.log
192.168.1.1 - - [26/Aug/2024:12:30:45 +0000] "GET /page1 HTTP/1.1" 200 1234
192.168.1.1 - - [26/Aug/2024:14:20:10 +0000] "POST /submit HTTP/1.1" 302 8765
# 192.168.1.1이 포함된 로그 줄만 출력됨
```

grep 옵션 활용

자주 쓰는 grep 옵션

```
$ grep -r "hello" .
# -r: 현재 디렉터리와 모든 하위 디렉터리를 재귀적으로 검색
$ grep -rn "hello" .
# -n: 매칭된 줄의 라인 번호까지 함께 표시
$ grep -ir "hello" .
# -i: 대소문자 구분 없이 검색 (Hello, HELLO도 매칭)
$ grep -r "t[wo]o" .
# 정규식 사용: t 다음 w 또는 o, 그 다음 o → "two", "too" 매칭
```

find + xargs + grep 파이프라인

조합 예시

```
$ find . -name "*.java" | xargs grep -i "hello"
```



.java 파일 목록을 찾음

파일명들을 받아
grep -i hello <file1> <file2> ... 형태로 실행

모든 .java 파일에서
"hello"(대소문자 무시)를 검색

핵심 grep -ir "hello" . 와 비슷하지만, find로 검색 대상을 더 정교하게 지정할 수 있습니다.

8

텍스트 처리 도구 더 보기: sed · awk

grep이 "찾기"에 특화되어 있다면, sed와 awk는 텍스트를 찾아서 바꾸거나 가공하는 데 사용합니다.

sed — Stream EDitor

```
$ cat file.txt
foo bar foo

$ sed 's/foo/bar/g' file.txt
bar bar bar
# 화면에 출력만 함 (file.txt 내용은 그대로)

$ sed -i 's/foo/bar/g' file.txt
# -i: file.txt 자체를 bar bar bar로 덮어씀
```

sed는 기본적으로 원본을 열어 고치지 않고, 한 줄씩 읽으며 바뀐 결과를 화면에 출력합니다. -i 옵션을 주면 그 결과로 원본 파일 자체를 덮어쓰므로, 실수에 대비해 sed -i.bak 처럼 백업 확장자를 함께 쓰는 것이 안전합니다(file.txt.bak에 원본).

awk — 필드 기반 처리

```
$ cat access.log
192.168.1.1 - - [26/Aug/2024:12:30:45 +0000] "GET /page1
HTTP/1.1" 200 1234

$ awk '{print $1}' access.log
192.168.1.1
# 공백 기준으로 나눈 첫 번째 필드(IP) 출력
```

필드(field) 분해 — 위 access.log 기준

$1	192.168.1.1 (IP)
$6~8	"GET /page1 HTTP/1.1" (요청)
$9	200 (상태 코드)
$NF	마지막 필드 (여기서는 1234, 응답 크기)

9

압축과 해제: tar

압축

```
$ tar -czvf archive.tar.gz  /path/directory
```



.tar.gz

-c 아카이브 생성 -z gzip으로 압축
-v 진행 상황 표시 -f 파일명 지정

압축 해제

```
$ tar -xzvf archive.tar.gz
# -x: 압축 해제(extract)
```

10

셸 스크립트 작성하기

셸이 직접 실행하도록 만든 프로그램(스크립트)입니다.

sample.sh

```
#!/bin/bash
target_dir="/path/to/directory"
output_file="archive.tar.gz"
find "$target_dir" -type f -name "*.txt" | xargs tar -czvf "$output_file"
```

하는 일 target_dir 안(하위 디렉터리 포함)에 있는 모든 .txt 파일만 골라 archive.tar.gz라는 압축 파일 하나로 묶습니다 — .jpg, .log 같은 다른 확장자 파일은 무시됩니다.

실행

```
$ chmod a+x sample.sh
# 실행 권한 부여
$ ./sample.sh
```

11

파일 다운로드 & 웹 통신: wget · curl

wget은 다운로드 전용 도구이고, curl은 다양한 프로토콜과 통신하는 범용 클라이언트입니다 — 다운로드는 curl 기능의 일부일 뿐입니다.

wget — 다운로드 전용 도구

HTTP·HTTPS·FTP에서 파일을 받아 저장하는 데 특화 — 기본 동작 자체가 곧 다운로드입니다.

curl — 범용 통신 클라이언트

Client URL의 약자 — HTTP·FTP·SMTP 등 다양한 프로토콜로 데이터를 주고받음. 다운로드는 -O로 켜는 기능 중 하나일 뿐입니다.

wget — 저장이 기본값

```
$ wget https://example.com/file.png
# 웹에서 파일을 그대로 다운로드해 저장

# 옵션 없이 실행해도 동작은 "다운로드" 하나뿐
```

curl — 저장은 옵션(-O)일 뿐

```
$ curl -O https://.../file.png
# -O: 파일로 저장 (다운로드 모드로 전환)

$ curl -I https://example.com
# 응답 헤더만 확인 — 파일을 받지 않고 "접속"만 함

$ curl -X POST -d '{"name":"kim"}' \
    https://api.example.com/users
# 서버로 데이터 전송 — 다운로드와 반대 방향
```

TIP Amazon Linux 2023 등 최신 배포판에는 curl이 기본 설치되어 있으나, wget은 별도로 dnf install wget이 필요할 수 있습니다.

12

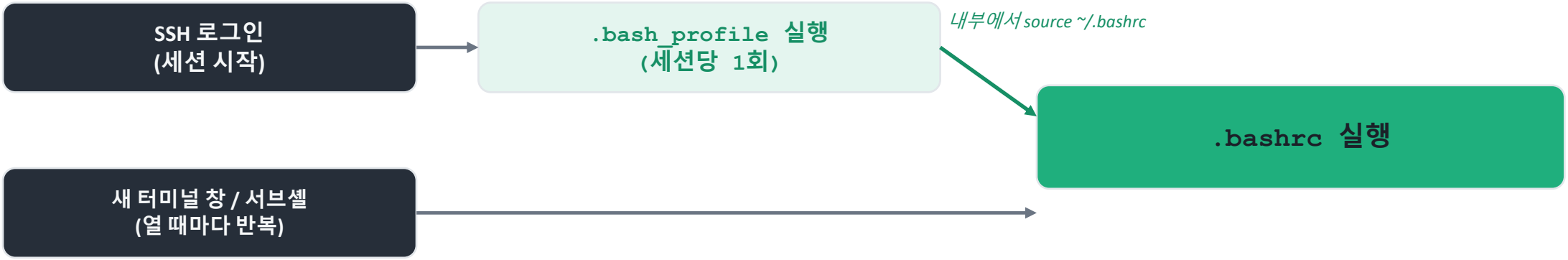
사용자 환경 설정: .bash_profile · .bashrc

.bash_profile

사용자가 로그인할 때 한 번 실행되며, 로그인 세션의 초기 설정을 구성합니다.

.bashrc

새 셸(터미널)을 열 때마다 실행되며, alias·PATH 등 개인화 설정에 사용됩니다.



핵심 로그인할 때는 .bash_profile → .bashrc 순서로 둘 다 실행되지만, 이후 새 터미널·서브셸에서는 .bashrc만 실행됩니다. (.bash_profile 안의 if [-f ~/.bashrc]; then . ~/.bashrc; fi 코드가 이 연결을 담당)

alias · PATH · 환경변수 커스터마이징

.bashrc 커스터마이징

```
# Add your own aliases here
alias ll='ls -alF'
alias myalias='echo This is my alias!'

# Additional paths to be added to PATH
if ! [[ "$PATH" =~ "/opt/myapp/bin:" ]]; then
    PATH="/opt/myapp/bin:$PATH"
fi
export PATH
```

환경변수란?

- PATH처럼 셸과 프로그램이 참조하는 이름-값 쌍의 설정 정보입니다.
- export로 지정하면 이후 실행되는 자식 프로세스에도 값이 전달됩니다.
- env 명령으로 전체 목록을 확인할 수 있습니다.

정리하기

1. 명령어는 이름 + 옵션 + 인자 구조이며, man으로 사용법을 확인하고 관리자 작업은 sudo로 실행합니다.
2. 표준 입출력(stdin/stdout/stderr)을 >, <, | 로 자유롭게 연결·저장할 수 있습니다.
3. cp/mv/rm/mkdir 등으로 파일을 다루고, ln -s(심볼릭)와 ln(하드) 링크의 차이를 구분합니다.
4. chmod로 권한을, chown/chgrp로 소유자·그룹을 관리합니다.
5. find·grep·sed·awk·xargs를 조합하면 대량의 텍스트/파일을 강력하게 처리할 수 있습니다.
6. tar로 압축하고, 셸 스크립트로 반복 작업을 자동화하며, curl/wget으로 파일을 내려받습니다.
7. .bash_profile · .bashrc · 환경변수로 나만의 작업 환경을 구성합니다.